

A Federated Learning Scheme with Adaptive Hierarchical Protection and Multiple Aggregation

Zhiqiang Wang^{*,†}

Department of Cyberspace Security, Beijing
Electronic Science and Technology Institute
Beijing, China
State Information Center
Beijing, China
wangzq@besti.edu.cn

Xinyue Yu[†]

Department of Cyberspace Security, Beijing
Electronic Science and Technology Institute
Beijing, China
757503836@qq.com

Ziqing Tian[†]

Department of Cyberspace Security, Beijing
Electronic Science and Technology Institute
Beijing, China
2194800501@qq.com

([†]These authors contributed to the work equally and should be regarded as co-first authors.)

Abstract—The results of federated learning contain certain private information and still have the risk of privacy leakage. Therefore, this paper proposes a Federated Learning Scheme with Adaptive Hierarchical Protection and Multiple Aggregation (ADDFed). The scheme trains models and generates updates using local datasets at the client side. Based on comparisons between client contributions and two thresholds, it chooses homomorphic encryption, adaptive differential privacy, or data discarding methods to protect the data. The first aggregation model and the second aggregation model are obtained by classification aggregation at the server side. Finally, the client decrypts and aggregates to get the global model. Experimental results demonstrate that ADDFed consumes a privacy budget of 3624.47, involves 562,219,358 bytes of communication, and requires 11,603 seconds of training time, achieving a model accuracy of 87.4%, thus showing significant advantages.

Keywords—Federated Learning, Privacy Protection, Homomorphic Encryption, Differential Privacy, Adaptive Privacy Budget

I. INTRODUCTION

In the current era of big data, each enterprise, and even different departments within the same enterprise, store vast amounts of application data. However, due to differences in data definition and organization, these have led to independent and difficult-to-correlate "data silos", hindering data flow and utilization. To address the issue of "data silos", Google proposed federated learning in 2016. This distributed machine learning method [1] trains models by sharing only model parameters without uploading local data [2], thereby ensuring data privacy and security. It allows data to flow in an "available but invisible" state, thereby helping to alleviate the problem of "data silos".

However, despite providing an effective approach to data privacy protection, federated learning still faces challenges related to privacy leakage. A malicious server may identify the data source by deducing the client uploading model information, thereby compromising data privacy. Similarly, a malicious client could compromise model confidentiality through attacks such as model extraction or reverse engineering, resulting in further data leakage. Therefore, addressing privacy protection issues while ensuring the efficacy of federated learning has become a significant research focus.

Currently, the main solutions to address this problem

include differential privacy, homomorphic encryption, and secure multiparty computation. These technologies process data exchange processes to achieve the goal of protecting data security. However, these methods may have adverse effects on model accuracy, training time, or communication volume. Therefore, proposing a new solution that achieves a balance in security, model accuracy, training time, communication volume, and other aspects is of significant importance for the application and development of federated learning.

In this paper, we focus on the privacy protection problem in federated learning and design a federated learning scheme with adaptive hierarchical protection and multiple aggregation (ADDFed) to achieve utility balance in federated learning. This method optimizes the balance of security, model accuracy, training time, and communication volume. Further research and analysis are conducted on both the federated learning training process and privacy-preserving algorithms. **The main contributions and innovations of this paper are as follows:**

- **Combining adaptive differential privacy and homomorphic encryption techniques for privacy protection.** This method introduces adaptive differential privacy and homomorphic encryption techniques. Based on the comparison between client contributions and thresholds, it selects differential privacy protection, homomorphic encryption, or direct discarding for local model updates. During the decryption aggregation preservation phase, accuracy and loss values are retained for adjusting the privacy budget of adaptive differential privacy in subsequent rounds.

- **Proposing a federated learning scheme with adaptive hierarchical protection and multiple aggregation.** After adaptive hierarchical protection, the server aggregates models separately based on different encryption methods. The client decrypts the second aggregation model and aggregates the decrypted result with the first aggregation model to obtain the global model trained in this round.

- **Analysis and evaluation of method performance.** This paper conducts a detailed analysis of the security of uplink and downlink data and experiments with logistic regression models on the MNIST dataset. The results indicate that compared to other federated learning algorithms, ADDFed reduces the privacy budget while maintaining

model accuracy, with shorter training times and less communication, demonstrating its superiority.

II. RELATED WORK

Literature[3] mentions the use of additive homomorphic encryption in longitudinal federated learning to achieve privacy protection for linear regression models, but it exposes the computation results of the loss function. Literature[4] employs CKKS homomorphic encryption to encrypt model parameters, ensuring the security of federated learning and addressing potential privacy data leakage issues. Despite its better communication and computational performance compared to schemes using Paillier homomorphic encryption, it still incurs additional time overhead compared to the federated averaging scheme. Literature[5] introduces FedMF, a secure matrix factorization framework for federated learning setups. Initially, the distributed MF method divides iterative updates into two parts executed at the user and server sides, respectively. Homomorphic encryption is subsequently employed to protect the factorization framework and user privacy. However, the use of homomorphic encryption introduces significant time overhead. Literature[6] proposes Dubhe, a pluggable system-level client selection method. This method allows clients to actively participate in training and employs homomorphic encryption to protect client privacy. However, it introduces additional communication overhead during new registration requirements and multiple selection processes.

Literature[7] utilizes homomorphic encryption to encrypt parameters transmitted by each client and utilizes secret sharing for key management and collaborative decryption to safeguard data privacy. However, the computational complexity associated with its use of homomorphic encryption remains high and requires further reduction to enhance operational efficiency. Literature[8] adopts secure multi-party computation to upload partial gradients and dynamically allocate compressed privacy budgets to improve model accuracy. This approach ensures data privacy and enhances model availability, but challenges persist in communication and computational aspects. Literature[9] proposes an efficient and privacy-preserving federated deep learning protocol based on stochastic gradient descent. The protocol integrates additive homomorphic encryption and differential privacy, achieving high efficiency and model accuracy. Next steps include optimizing computational overhead through a multi-key encryption scheme. Literature[10] introduces HybridAlpha, an efficient federated learning framework that combines secure multi-party computation and differential privacy. This scheme reduces training time and data transmission without compromising security and model performance. Literature[11] presents a privacy-preserving federated learning scheme based on homomorphic encryption and differential privacy. The scheme applies differential privacy to scramble parameters transmitted by each client before encrypting them using Paillier's algorithm to protect data privacy. However, the computational overhead due to the use of Paillier's algorithm remains substantial. Literature[12] proposes a secure aggregation protocol that combines homomorphic encryption with secret sharing in federated learning. Symmetric homomorphic encryption is used to encrypt transmission parameters to facilitate data privacy protection, while secret sharing techniques address user

dropout issues. Symmetric homomorphic encryption resolves the high cost associated with homomorphic encryption under a public key system. Literature [13] introduces a multi-server federated learning scheme that combines differential privacy and secret sharing by injecting noise into transmitted gradients and subsequently uploading these perturbed gradients to multiple servers through a secret sharing protocol. While this scheme ensures secure gradient transmission, communication overhead increases with the number of servers.

Based on an analysis of the current research landscape, cryptography-based protection schemes commonly employ techniques such as homomorphic encryption, secure multiparty computation, and secure aggregation to handle transmission data in federated learning. However, challenges remain regarding the high computational overhead and long training times associated with homomorphic encryption, as well as the significant communication costs of secure multiparty computation. Perturbation-based protection schemes primarily introduce noise to disturb transmitted data; failure to offset this added noise adequately can reduce data availability and consequently impact model accuracy. Differential privacy, as a frequently used approach, often fixes the privacy budget during the training process, which can lead to privacy leakage risks after multiple training rounds. Hybrid schemes, while combining the strengths of various techniques, still require exploration to achieve a balance among security, communication overhead, training time, and model accuracy.

III. METHODOLOGY

A. Methodology Overview

This section describes the specific implementation of the Adaptive Hierarchical Protection Multiple Aggregation Federated Learning Scheme (ADDFed), the main objective of which is to dynamically adjust the privacy budget and process the local model updates that choose to use differential privacy protection, which ultimately improves the ability to protect the data, increase the accuracy of the model, and complete the training using less training time and communication. The scheme is divided into five phases: initialization phase, local training phase, adaptive hierarchical protection phase, classification aggregation phase with decryption aggregation storage phase. The specific process is shown in Figure 3.1.

In the initialization phase, the key generation center generates the keys required for homomorphic encryption and distributes them to the clients through a secure channel. At the same time, the server establishes a connection with the client, selects the client that participates in the first round of training, and sends the initial global model parameters.

In the local training phase, the client independently completes the local training acquisition stage. The main purpose of this stage is for each client to train a local model update and obtain a local model based on their local dataset using the stochastic gradient descent (SGD) algorithm. Firstly, clients participating in the current round of training train their local models based on their respective local datasets, obtaining new local models. Next, they compute the difference between the new local model and the previous round's local model to derive the local model update for the current round. Subsequently, they obtain the local model update and the local model for further use.

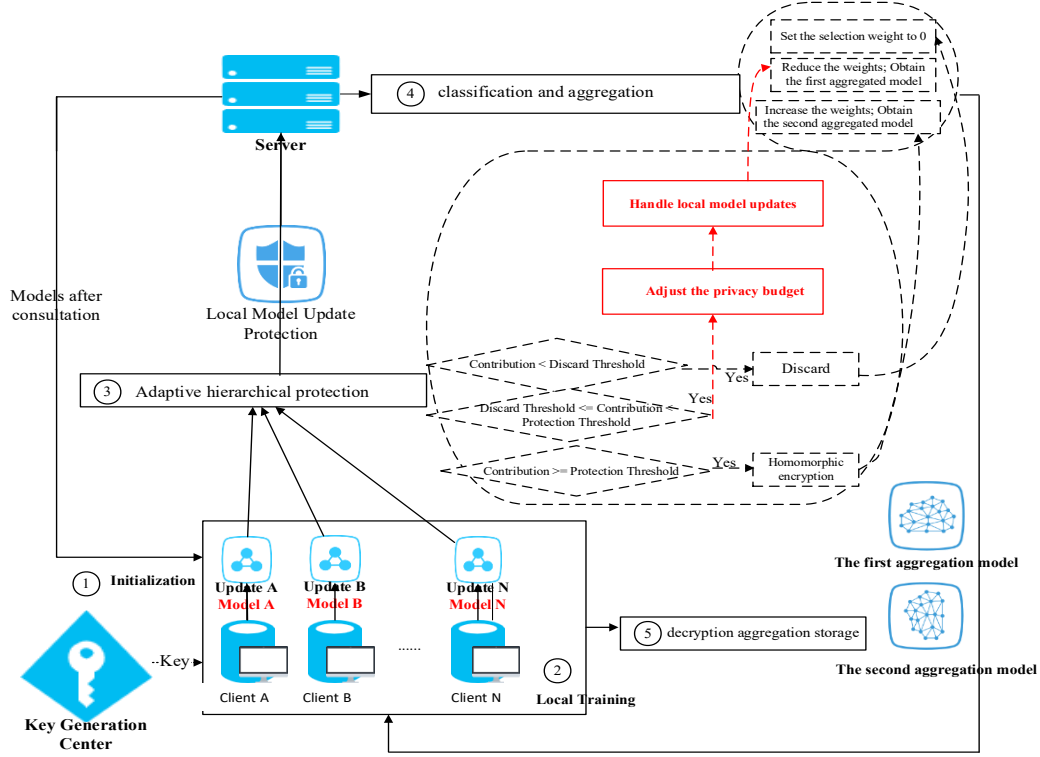


Figure 3.1 ADDFed Scheme Workflow

In the adaptive hierarchical protection phase, it is necessary to calculate client contributions and evaluate the relationship between thresholds and contributions to select protection methods such as homomorphic encryption, adaptive differential privacy, or discarding, and set flags for upstream data transmission. If adaptive differential privacy is chosen, coefficients are calculated and scoring results are adjusted to modify the privacy budget. The specific methods for this phase will be detailed in Section III.B.

In the classification and aggregation phase, adjustments are made to the client selection weight table. Clients using differential privacy decrease their selection weights, while clients using homomorphic encryption increase their selection weights. The first and second aggregation models are then classified and aggregated, and the next round of participating clients is selected to prepare for downstream data transmission and recipients.

In the decryption aggregation storage phase, clients decrypt the second aggregation model using corresponding homomorphic decryption algorithms, aggregate it with the first aggregation model, and obtain the plaintext global model trained in the current round. They save accuracy and loss values from testing the model on a test set for subsequent adjustments of the privacy budget when using differential privacy. The specific methods for this phase will be detailed in Section III.C.

This approach can be implemented using Algorithm 1, as shown in Table 3.1, where the specific contents of **ADJUST()** and **PROL()** are described in Section III.B.3, and the specific contents of **DEGRE()** are detailed in

Section III.C.

Table 3.1 The Implementation Process of ADDFed

Algorithm 1: ADDFed

Inputs: number of training times T , initial global model A^0 , clients $C = \{C_1, C_2, \dots, C_M\}$, number of clients participating in training N , dataset $D = \{D_1, D_2, \dots, D_M\}$, learning rate η , client selection weight *list*, discard threshold P_1 , protection threshold P_2 , weight value W_1 reduced using adaptive differential privacy, weight value W_2 increased using homomorphic encryption, key S , initial privacy budget ϵ .

Output: global model A^T

01 **Begin**

Client-side execution:

/* initialization phase */

02 Distribute S to $C = \{C_1, C_2, \dots, C_M\}$

03 Negotiate the client models involved in training
 $B_1^0, B_2^0, \dots, B_N^0 = A^0$

/* Local training acquisition phase */

04 **For** $t \leftarrow 1$ **to** T **do**

05 **For** $i \leftarrow 1$ **to** N **do**

06 $B_i^t \leftarrow \text{SGD}(A_i^{t-1}, \eta, D_i)$

07 $\Delta B_i^t \leftarrow A_i^{t-1} - \text{SGD}(A_i^{t-1}, \eta, D_i)$

/* Adaptive hierarchical protection stage */

/* Determining the size of the threshold in relation to the size of the contribution */

```

08   $coC_i^t \leftarrow (\text{same}(\text{sign})) / (\text{num}(\text{sign}))$ 
09  If  $coC_i^t < P_1$ 
10     $f_i^t \leftarrow 0$ 
11     $B_i^t \leftarrow \text{null}$ 
12  Elseif  $P_1 \leq coC_i^t < P_2$ 
13     $f_i^t \leftarrow 1$ 
14   $\varepsilon_i^t \leftarrow \text{ADJUST}(M, N, D_i, D, B_i^t, A_i^{t-1}, t, T, \text{acc}, \text{loss})$ 
15   $B_i^t \leftarrow \text{PROL}(B_i^t, A_i^{t-1}, \Delta B_i^t, \Delta f, \varepsilon_i^t)$ 
16  Else
17     $f_i^t \leftarrow 2$ 
18     $e \leftarrow \text{NP.RANDOM.RAND}(\text{LEN}(\Delta B_i^t))$ 
19     $B_i^t \leftarrow S^{-1} \cdot (W \times \Delta B_i^t + e)$ 
20  EndIf
21  Send uplink transmission data  $\{B_i^t, f_i^t\}$  to server
22  EndFor
Server execution:
/* classification aggregation stage */
23  For  $i \leftarrow 1$  to  $N$  do
24    If  $f_i^t == 0$ 
25       $\text{list}[i] \leftarrow 0$ 
26    Elseif  $f_i^t == 1$ 
27       $Q_1^t \leftarrow Q_1^t + B_i^t$ 
28       $\text{list}[i] \leftarrow \text{list}[i] - W_1$ 
29       $\text{num}^t \leftarrow \text{num}^t + 1$ 
30    Else
31       $Q_2^t \leftarrow Q_2^t + B_i^t$ 
32       $\text{list}[i] \leftarrow \text{list}[i] + W_2$ 
33       $\text{num}^t \leftarrow \text{num}^t + 1$ 
34    EndIf
35  EndFor
36  Based on the updated  $\text{list}$ , we select the clients
  to participate in the next round of training and downlink
   $Q_1^t, Q_2^t, \text{num}^t$ 
Client-side execution:
/* decryption aggregation stage */
37   $A^t \leftarrow \text{DEGRE}(Q_1^t, Q_2^t, S, \text{num}^t)$ 
38  EndFor
39  Save  $\text{acc}$  and  $\text{loss}$ 
40  Return  $A^T$ 
41  End

```

B. Adaptive Hierarchical Protection Phase

The adaptive hierarchical protection phase is the most important phase in the method, so it is described in detail here. This phase is mainly done by the client, first the client contribution is calculated, then the contribution needs to be first compared in size with the preset threshold, then based on the comparison result the homomorphic encryption or adaptive differential privacy or discarded is selected and tokenized to facilitate the acquisition of the local model update protection. After that, the local model update protection and its token are uploaded to the server.

B.1 Selection of model update protection methods

We first calculate the client contribution and select the protection method. The contribution degree is calculated as the ratio of the local model update of the client to the global model symbols in the same proportion. Then the contribution degree needs to be compared in size with a preset threshold first, and then the homomorphic encryption or adaptive differential privacy or discard method is selected and labeled according to the comparison result. Local model update protection in this scheme refers to the result after the local model update is protected. The process is specifically as follows:

Step 1: Calculate client contribution. In this method, client contribution is computed as the proportion of local model updates that match the global model's signs. The calculation formula is shown in Equation 3-1, where con represents client contribution, $\text{same}(\text{sign})$ denotes the number of local model updates matching the global model signs, and $\text{num}(\text{sign})$ indicates the total number of global model signs. For example, if the signs of local model updates are $(-, +, -, -)$ and the global model signs are $(-, -, -, -)$, the first, third, and fourth positions have matching signs, resulting in a client contribution of $3/4$.

$$con = (\text{same}(\text{sign})) / (\text{num}(\text{sign})) \quad (3-1)$$

Step 2: Obtain local model update protection. This step involves comparing the client contribution against predefined thresholds, selecting a protection method based on the comparison result, and applying corresponding markers. In this method, local model update protection refers to the outcome after protecting the local model updates.

Local model updates are handled based on their contribution: updates below the discard threshold are dropped (marker 0), those between the discard and protection thresholds are protected with differential privacy (marker 1), and those above the protection threshold are encrypted using homomorphic encryption (marker 2). This design prioritizes efficiency and security: updates below the discard threshold have negligible or negative impact on global model updates and are discarded to reduce communication overhead. Updates between thresholds are protected with noise addition, balancing security and minimal impact. Updates above the protection threshold, critical to global model performance, are encrypted to ensure both security and data utility.

B.2 Homomorphic Encryption Scheme

In order to achieve faster training speeds and reduced communication overhead, this method employs a vector homomorphic encryption scheme VHE[14], which offers lower computational complexity. The encryption formula is shown in Equation 3-2, where S represents the key, c is the ciphertext, x is the plaintext, W is a large integer, and e is random noise primarily used to ensure that deciphering the original message without the key is difficult, and can be rounded off when the key is available.

$$Sc = Wx + e \quad (3-2)$$

Given the characteristics of data transmission and the feasibility of implementation analysis, the differential privacy mechanism chosen for this method is the commonly used Laplace mechanism. Its output is expressed as shown in Equation 3-3, where D denotes the dataset, q represents the query, ε is the privacy budget, Δf is the sensitivity, and

$\text{Lap}(0, \Delta f/\varepsilon)$ indicates noise data following a Laplace distribution, with a location parameter of 0 and a scale parameter of $\Delta f/\varepsilon$.

$$M = q(D) + \text{Lap}(0, \Delta f/\varepsilon) \quad (3-3)$$

B.3 Adaptive Differential Privacy Scheme

In this scheme, the adaptive differential privacy method involves first adjusting the privacy budget before processing the local model.

Adjusting the privacy budget is divided into three steps: calculating the adjustment factor, calculating the scoring result based on the scoring function, and judging and adjusting:

Step 1: Calculate the adjustment coefficient. The adjustment coefficient is used to judge how to adjust the privacy budget of each client, which is calculated with respect to the cosine value of the global model and the local model, the number of client datasets, the total number of datasets, the number of selected clients and the total number of clients. The calculation formula is shown in Equation 3-4, where p is the adjustment coefficient, A is the global model of the previous round, B is the local model of the current round, M is the number of all the clients in this training, N is the number of selected clients in the current round of training, D_i is the number of datasets of this client, and D is the total number of datasets.

$$p = \begin{cases} 1 & \cos(A, B) < 0 \\ 1 - \cos(A, B) \times \frac{M}{N} \times \frac{D_i}{D} & \cos(A, B) \geq 0 \end{cases} \quad (3-4)$$

When the cosine value of the local model of the current round and the global model of the previous round is non-negative, it means that the two are more similar, and the more similar they are, the more noise needs to be added to protect them, so the privacy budget is made smaller by decreasing the adjustment factor. If the cosine value of the local model in this round and the global model in the previous round is negative, it indicates that the two are less similar and the more noise will make the data availability lower, so the adjustment factor is set to 1 not to change the initial privacy budget.

Step 2: Calculate the scoring results. The scoring results are used for subsequent adjustment of the privacy budget, and the calculation of the scoring results is related to the number of training rounds, the model accuracy and the loss value. The calculation formula is shown in Equation 3-5, where $socre$ is the scoring result, $socre_{loss}$ is the scoring result of loss value, $socre_{acc}$ is the scoring result of accuracy rate, and $socre_t$ is the scoring result of training rounds.

$$socre = 30socre_{loss} + 40socre_{acc} + 30socre_t \quad (3-5)$$

Specific calculations of the parameters therein are as follows:

(1) Loss value. The loss value is the error between the output value and the actual value [15], which should gradually become smaller and eventually keep floating in a small range during the training process. Therefore, the scoring formula of loss value is defined as shown in Equation 3-6, where $loss_i$ is the loss value of the i th round, and $loss_{i-1}$ is the loss value of the $i-1$ round.

$$socre_{loss} = \begin{cases} 1 & loss_i \geq loss_{i-1} \\ 0 & loss_i < loss_{i-1} \end{cases} \quad (3-6)$$

If the loss value in round $i-1$ is greater than in round i , the training is progressing normally, and the score is set to 1, increasing the overall score to lower the privacy budget and enhance data protection. Conversely, if the loss value in round $i-1$ is less than in round i , indicating difficulty in reducing the loss, the score is set to 0 to maintain the privacy budget and focus on further reducing the loss.

(2) Accuracy. The accuracy rate should trend upwards during the training process and eventually maintain a more stable state. Since the fluctuation of the accuracy rate is more obvious than the fluctuation of the loss value in general and there is a big difference between the accuracy rates of two adjacent rounds, this algorithm averages the accuracy rates and then compares them. Therefore, the scoring formula for defining the accuracy rate in this algorithm is shown in Equation 3-7, where acc_j is the accuracy rate of the j th round, the i th round is the last round, and N is the number of clients selected in this training. The formula is related to the number of selected clients N , for example: if the number of selected clients in this training is 3, in the 10th round, the average accuracy rate is calculated as $(acc_9 + acc_8 + acc_7)/3$.

$$socre_{acc} = \begin{cases} 1 & \sum_{j=i-N+1}^i acc_j / N \geq acc_i \\ 0 & \sum_{j=i-N+1}^i acc_j / N < acc_i \end{cases} \quad (3-7)$$

If the average accuracy is greater than the current round's accuracy, the accuracy has basically stabilised, so the value is defined as 1, which increases the scoring so that the privacy budget is lower and the ability to protect the data is improved. If the average accuracy rate is smaller than the current round accuracy rate, the accuracy rate of this round still has room to increase, so the value is defined as 0 to make the scoring result smaller so that the privacy budget is not changed and the accuracy rate is further improved.

(3) Training rounds. Under normal circumstances, as the number of training rounds increases, the accuracy rate will become higher and the loss value will decrease. Therefore define the scoring formula for training rounds as in Equation 3-8, where t is the current training round and T is the total training rounds.

$$socre_t = 2\min\left(\frac{t}{T}, \frac{1}{2}\right) \quad (3-8)$$

As the number of training rounds increases, the degree of change in accuracy and loss values gradually decreases, and the risk of privacy leakage gradually increases, so the scoring of the training rounds is gradually increased to the final state of maintaining a perfect score, so that the privacy budget is reduced as much as possible, thus improving the ability to protect the data as much as possible.

Step 3: Judgement and adjustment. Judge the adjustment method of privacy budget according to the adjustment coefficient and scoring results. The judgement and adjustment process is shown in Equation 3-9. Where ε' is the adjusted privacy budget, ε is the initial privacy budget, p is the adjustment coefficient, and $socre$ is the scoring result.

$$\varepsilon' = \begin{cases} p\varepsilon & socre > 50 \text{ and } p \leq 1 \\ \varepsilon & socre \leq 50 \text{ or } p > 1 \end{cases} \quad (3-9)$$

Combined with the above, the reason for the privacy budget adjustment is because:

When the scoring result is less than or equal to 50 [16], then it indicates that fewer training rounds local model updates are more variable not easy to infer the original data, or the accuracy and loss values are stuck in a bottleneck, which is difficult to change in a positive direction. Therefore do not change the privacy budget, adding less noise thus ensuring data availability and promoting model convergence. $p > 1$ occurs with a high probability because the number of client selections as a proportion of the total number of clients is too small, the fewer the clients trained the lower the accuracy of the model, so do not change the privacy budget, as far as possible to ensure model accuracy.

When the scoring result is greater than 50 [16], it means that more training rounds local model updates are less variable easier to infer the original data, or the accuracy and loss values are still changing in a positive direction. Therefore more noise needs to be added to protect the data. $p \leq 1$ occurs as a common case, which is a prerequisite for lowering the privacy budget, and multiplying it with the privacy budget achieves enhanced data protection.

Processing the local model is divided into three steps: calculating the scaling factor, generating noise, and scaling and adding noise, as follows:

Step 1: Calculate the scaling factor. The scaling factor is obtained based on the cosine similarity between the global model and the local model in the previous round, and the calculation formula is shown in Equation 3-10. Where ∂ is the scaling factor, A is the global model of the previous round, and B is the local model of the corresponding client in this round.

$$\partial = \cos(A, B) \quad (3-10)$$

The larger the cosine value of the local model of the current round and the global model of the previous round, the more similar the two models are, and the larger the scaling factor, the more changes are made to make the local model update. The smaller the cosine value, the less similar the two models are, and the smaller the scaling factor, the smaller the change to make the local model update, ensuring data availability as much as possible.

Step 2: Generate noise. This method is based on the Laplace mechanism, which replaces setting a fixed privacy budget with setting an adjusted privacy budget, and the adjustment method is described in 3.4.2. The noise generation formula is shown in Equation 3-11, where *noisy* is the generated noise, ϵ' is the adjusted privacy budget, Δf is the sensitivity, $\text{Lap}(0, \Delta f / \epsilon')$ represents the data noise that obeys the Laplacian distribution, and the location parameter of this Laplacian distribution is 0 and the scale parameter is $\Delta f / \epsilon'$.

$$\text{noisy} = \text{Lap}(0, \Delta f / \epsilon') \quad (3-11)$$

Step 3: Scaling and noise addition. The local model update before uploading is scaled and then noisy, as shown in Equation 3-12, where G' is the uploaded noisy local model update, G is the local model update before uploading, ∂ is the scaling factor, and *noisy* is the generated noise.

$$G' = \partial G + \text{noisy} \quad (3-12)$$

This section processes the local model updates both in

terms of itself and noise, ensuring both data availability and security. The detailed process of processing the local model updates can be represented as an algorithm, as shown in Table 3.2, which corresponds to **PROL()** in Table 3.1.

Table 3.2 ADDFed implementation process for processing local models

Algorithm 2: Processing Local Model Updates

Input: adjusted privacy budget ϵ' , sensitivity Δf , global model A_i^{t-1} for round $t-1$, local model B_i^t for the client i in round t , and local model update ΔB_i^t for the client i in round t .

Output: local model update Q_i^t for the transmission of the client i in the round t .

01 **Begin**

/* Calculating the scaling factor */

// Calculating cosines

$$02 \quad \partial \leftarrow \frac{A_i^{t-1} \cdot \text{DOT}(B_i^t)}{NP.LINALG.NORM(A_i^{t-1}) * NP.LINALG.NORM(B_i^t)}$$

/* generate noise */

$$03 \quad \text{noisy} \leftarrow \text{LAP}(0, \Delta f / \epsilon')$$

/* Scaling and Noise Addition */

$$04 \quad Q_i^t \leftarrow \partial * \Delta B_i^t + \text{noisy}$$

05 **End**

C. Decryption Aggregation Storage Phase

The decryption aggregation storage phase is primarily conducted by clients and includes decrypting the second aggregated model and aggregating it with the first aggregated model to obtain the current global model for the next round of local models. The specific steps are as follows:

Step 1: Decrypt the second aggregated model. After receiving the first and second aggregated models, clients decrypt the second aggregated model using the chosen homomorphic encryption algorithm and the corresponding key. Given that symmetric encryption is used in this method, the decryption formula is shown in Equation 3-13.

$$x = \left[\frac{Sc}{W} \right] \quad (3-13)$$

Step 2: Obtain the global model. Clients aggregate the decrypted second aggregated model with the first aggregated model to obtain the global model for the current round of training, which also serves as the initial local model for the next round. At the end of training, this becomes the final global model. The aggregation formula is shown in Equation 3-14, where A^t represents the global model for the t round, Q_1^t is the first aggregated model for the t -th round, Q_2^t is the decrypted second aggregated model for the t round, and num^t is the total number of clients contributing protected local model updates through differential privacy and homomorphic encryption.

$$A^t = (Q_1^t + Q_2^t) / num^t \quad (3-14)$$

The detailed process of the decryption aggregation storage phase can be algorithmically represented, as shown in Table 3.3, where the specific content of deriving the latest global model corresponds to the details in Table 3.1 under **DEGRE()**.

Table 3.3 Implementation Process of the Decryption and Aggregation Phase

Algorithm 3: Decryption and Aggregation Phase

Input: The global model A^{t-1} from iteration $t-1$, the first aggregated model Q_1^t from iteration t , the second aggregated model Q_2^t from iteration t , the total number of clients num^t using homomorphic encryption and differential privacy for local model updates in iteration t , a large integer W , and a key S .

Output: The global model A^t for iteration t .

01 **Begin**

/* Decrypt and aggregate to obtain the latest global model **DEGRE()** */

02 $Q_2^t \leftarrow [S \cdot Q_2^t / W]$ // Decrypt

03 $A^t \leftarrow A^{t-1} - (Q_1^t + Q_2^t) / num^t$ //Aggregate

04 **Return** A^t

05 **End**

Step 3: Save model accuracy and loss values. The global model for this round is tested using the test set, and the resulting model accuracy and loss values are saved for use in later rounds when adjusting the privacy budget if adaptive differential privacy is selected in the adaptive hierarchical protection phase.

IV. PROOF OF SECURITY

This section analyzes the security of ADDFed and analyzes the security of the data with respect to the design goals, specifically the security of the local model update during uplink transmission and the security of the global model during downlink transmission in the server.

Theorem 4.1: If the data transmitted upstream in ADDFed, i.e., the local model update protection, is proved to be secure, then the local model update protection prevents the leakage of the original local model update to internal and external adversaries.

PROOF: This scheme chooses the method of adaptive hierarchical protection for the uplink transmitted data.

The specific process is shown in Equation 4-1, where C_i is the contribution degree of the i th client, α is the discard threshold, β is the protection threshold, $G(D_i) = (g_i^1, g_i^2, \dots, g_i^n)$ is the scaled local model update of the i th client, $Q(D_i)$ stands for the protection of the final transmission of the local model update, ϵ_i is the adjusted privacy budget.

$$Q(D_i) = \begin{cases} \text{null}, C_i < \alpha \\ G(D_i) + \text{Lap}\left(0, \frac{\Delta f}{\epsilon_i}\right), \alpha \leq C_i < \beta \\ S^{-1} \cdot (W * G(D_i) + e), C_i \geq \beta \end{cases} \quad (4-1)$$

Since the ranges of $C_i < \alpha$, $\alpha \leq C_i < \beta$, and $C_i \geq \beta$ are mutually independent, the security of local model updates depends on three mechanisms: non-transmission of the updates, the addition of adaptive noise, and the use of VHE encryption for local model updates. The VHE algorithm provides semantic security, and the improved Laplace mechanism satisfies ϵ_i -differential privacy.

During uplink transmission, there are three possible scenarios: (1) No transmission, making it impossible for attackers or the server to access the original local model

update; (2) Transmission of local model updates with adaptive noise, where attackers and the server can only access the noisy updates and cannot recover the original model; (3) Transmission of encrypted local model updates, where attackers and the server can only access the ciphertext, but since the key is stored only on the client side, they cannot decrypt it and thus cannot access the original local model update.

Therefore, the analysis yields that the adaptive hierarchical protection phase during uplink transmission makes the attacker and server unable to obtain the original local model update and the original local model update is secure.

Theorem 4.2: If the data transmitted downstream in ADDFed, i.e., the first aggregation model and the second aggregation model, are provably secure, then the first aggregation model and the second aggregation model can prevent the global model from being leaked to internal and external adversaries.

PROOF: This scheme selects multiple aggregation, i.e., categorical aggregation and decryptive aggregation, for the protection of data transmitted downstream.

The process of classification aggregation is shown in Equation 4-2, and the process of decryption aggregation is shown in Equation 4-3, where q is the total number of clients using the improved LAP in this round, r is the total number of clients using the VHE in this round, m_1 is the first aggregation model, m_2 is the second aggregation model, and model is the global model.

$$\begin{cases} m_1 = \sum_{j=1}^q (G(D_j) + \text{Lap}\left(0, \frac{\Delta f}{\epsilon_j}\right)) \\ m_2 = \sum_{z=1}^r (S^{-1} \cdot (W * G(D_z) + e_z)) \end{cases} \quad (4-2)$$

$$\text{model} = \frac{m_1 + \frac{S m_2}{W}}{q + r} \quad (4-3)$$

The server aggregates the uplink-transmitted local model updates with adaptive noise and encrypted updates separately. The first aggregated model, m_1 , is obtained by aggregating the noisy local model updates, while the second aggregated model, m_2 , is obtained from the encrypted updates. The global model is derived by decrypting m_2 on the client side and combining it with m_1 .

The improved Laplace mechanism preserves post-processing invariance, meaning that additional computations on differentially private results still yield differentially private outcomes, so m_1 remains differentially private. Using the homomorphic addition property of VHE, m_2 remains encrypted, and only the client, with the decryption key S , can decrypt it. Since S is kept on the client side, neither the server nor the attacker can decrypt m_2 . Therefore, during server-side processing and downlink transmission, the server and attacker can only access m_1 and m_2 , but cannot obtain the decrypted value of m_2 , and thus cannot aggregate them to derive the global model.

Therefore, the analysis leads to the conclusion that in the server and for downstream transmission, the categorical aggregation and decryption aggregation makes it impossible for the attacker and the server to obtain the global model in plaintext and the global model is secure.

V. EXPERIMENTAL EVALUATION

A. Experimental environment

The experimental environment is shown in Table 4.1.

Table 4.1 ADDFed experimental environment

Name	Parameter information
Hardware Information	NVIDIA GPU GeForce RTX 3090 32GB of Memory
Software Information	Ubuntu 20.04 LTS Python 3.7 PyTorch 1.7.1 Anaconda Numpy Pandas
GPU Components	NVIDIA GPU Driver CUDA 11.1

B. Performance experiments

The focus of this paper is on the security of the transmitted data, and the results of the experiments are less related to the dataset chosen for the experiments itself, so we chose the widely used MNIST dataset, which facilitates the implementation of the simulation experiments. This scheme involves thresholds, so experiments are conducted with different thresholds and the results are analyzed to discuss the performance of the scheme in terms of feasibility, total privacy budget, model accuracy, communication volume, training time, and other factors as follows:

The experiments can be categorized into two groups based on the different ways of dividing the dataset: (1) Label-based unbalanced dataset Equal: each client has the same amount of data, but different numbers of labels. (2) Quantity-based unbalanced dataset Random: each client has different numbers of data and labels. Meanwhile, the number of fixed experimental rounds is 1,200, the learning rate is 0.1, the batch size is 128, and both the weight reduction and increase values set at 500. The specific experimental parameters are shown in Table 4.2. The two threshold parameters were manually selected and experimented with multiple times.

Table 4.2 ADDFed Experimental parameters

groups	serial number	P_1	P_2
Equal	1-1	0	0.5
	1-2	0.41	0.55
	1-3	0.41	0.45
	1-4	0.31	0.5
	1-5	0.4	0.5
	1-6	0.41	0.5
	1-7	0.42	0.5
Random	2-1	0.43	0.5
	2-2	0.44	0.5
	2-3	0.45	0.5
	2-4	0.44	0.49

The feasibility of this approach is validated by the accuracy and loss curves observed during training. As shown in Figures 4.1 and 4.2, both the Equal and Random groups demonstrate increasing accuracy that stabilizes within a narrow range, while loss curves consistently decline and stabilize with minor fluctuations. These normal variations and stable final fluctuations confirm model convergence and the approach's feasibility.

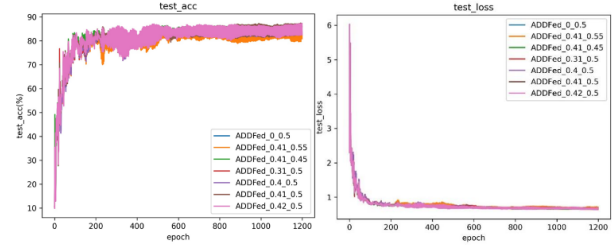


Figure 4.1 Equal group test curve results

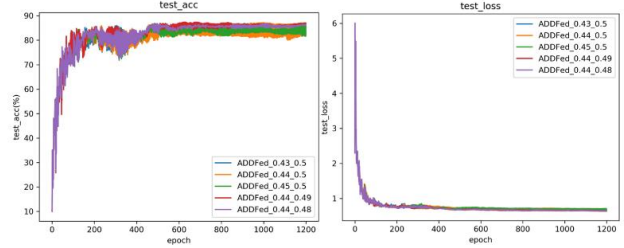


Figure 4.2 Random group test curve results

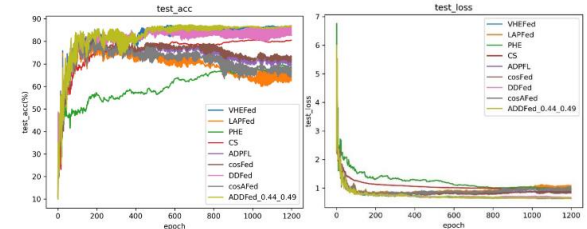


Figure 4.3 Results of ADDFed comparison experiment curves

Table 4.3 Results of ADDFed comparison experiments

Method	Total privacy budget	Best_acc (%)	Communication Volume(bytes)	Training time (seconds)
VHEFed	-	87.14	560821165	14549
LAPFed	360000	80.07	610592565	1092
PHE	-	69.72	20541775916	275110
CS	-	80.66	621205978	2599
ADPFL	353475	81.23	619384747	1243
cosFed	376443.70	81.55	619921704	1219
DDFed	40500	86.61	564137651	12650
cosAFed	305972.81	80.85	617619795	1218
ADDFed _0.44 _0.49	3624.47	87.4	562219358	11603

This scheme considers the number of datasets owned by the client when adjusting the privacy budget, which is more closely related to the dataset division method used by the Random group. Therefore, in this section, the Random group is selected for comparison with ADDFed, VHE-based federated learning algorithm (VHEFed), LAP-based federated learning algorithm (LAPFed), a secure and

efficient homomorphic encryption-based federated learning privacy-preserving scheme (PHE) [17], an effective privacy-preserving federated learning scheme based on compression sensing (CS) [18], optimization study of differential privacy based federated learning algorithm (ADPFL) [16], local differential privacy design for federated learning (cosFed) [19], a federated learning scheme for hierarchical protection and multiple aggregation(DDFed) [20], and an adaptive differential privacy method based on federated learning(cosAFed) [21]. DDFed and cosAFed are compared under the same experimental conditions, with a discard threshold set at 0.44 and a protection threshold set at 0.49. The experiments are conducted with 1,200 fixed training rounds, 3 clients selected, a learning rate of 0.1, a batch size of 128, and both the weight reduction and increase values set at 500. The experimental results are presented in Figure 4.3 and Table 4.3.

Comparing Figure 4.3 and Table 4.3, the nine schemes are ranked by privacy budget, model accuracy, training time, and communication traffic as follows: (1) Among six schemes with a privacy budget, ADDFed ranks first with the smallest budget of 3624.47. (2) By model accuracy, ADDFed ranks highest at 87.4%. (3) By communication volume, ADDFed ranks second, 1,398,193 bytes more than the first but 19,979,556,558 bytes less than the last. (4) By training time, ADDFed ranks sixth, 10,511 seconds longer than the first but 263,507 seconds shorter than the last.

Therefore, compared to other schemes, this approach demonstrates certain advantages. With a minimal privacy budget of 3624.47 and achieving the highest model accuracy of 87.4%, ADDFed performs notably well. It also manages communication efficiently with a volume of 562,219,358 bytes, completing training in only 11,603 seconds.

VI. CONCLUSION

ADDFed selects adaptive differential privacy techniques, homomorphic encryption techniques, or discarding to process local model updates based on the comparison results between client contributions and preset thresholds. On the server side, it adjusts the probabilities of clients being selected and aggregates the noisy model and encrypted model respectively. Finally, clients decrypt the encrypted model and aggregate it with the noisy model to obtain the global model. Comparative experiments show that ADDFed only takes 11603 seconds, with the lowest privacy budget of 3624.47, achieving the highest model accuracy of 87.4% and has the second lowest communication traffic of 562219358 bytes. Therefore, ADDFed improves security while increasing model accuracy and reducing training time and communication traffic.

In the future, this paper will conduct research in the following areas: (1) reducing the communication traffic of downstream transmission to decrease the total communication traffic of both upstream and downstream transmissions; (2) automating the selection of thresholds to save human and financial resources.

Acknowledgment

This research was supported by the Fundamental Research Funds for the Central Universities (Grant No. 3282024021, 3282024050, 20230045Z0114), First-class Discipline Construction Project of Beijing Electronic Science and Technology Institute (Grant No. 3201012).

References

- [1] Sheng, X. Research on Text Classification Models Based on Distributed Machine Learning [Master's thesis, Nanjing University of Posts and Telecommunications]. DOI: 10.27251/d.cnki.gnjdc.2023.001245
- [2] Wu, J, Li, X. Segmented cropping federated learning algorithm based on differential privacy[J/OL]. Computer Applications Research:1-7[2024-0111]. <https://doi.org/10.19734/j.issn.10013695.2023.09.0402>.
- [3] Yang Q, Liu Y, Chen T, Tong Y. Federated machine learning: Concept and applications. ACM Transactions on Intelligent Systems and Technology (TIST). 2019, 10(2):1-19.
- [4] QIU F Y, YANG H, ZHOU H, et al. Privacy preserving federated learning using CKKS homomorphic encryption[J]. Wireless Algorithms, Systems, and Applications, 2022
- [5] Chai D, Wang L, Chen K, et al. Secure Federated Matrix Factorization[J]. Intelligent Systems, IEEE, 2020, PP(99):1-1.
- [6] ZHANG S, LI Z, CHEN Q, et al. Dubhe: towards data unbiasedness with homomorphic encryption in federated learning client selection[C]// Proceedings of the 50th International Conference on Parallel Processing, Lemont, Aug 9-12, 2021. New York: ACM. 2021: 1-10.
- [7] Lai, C., Zhao, Y., & Zheng, D. (2024). Privacy Protection and Verifiable Federated Learning Scheme Based on Homomorphic Encryption. *Information Network Security*, 24(01), 93-105.
- [8] Gong M, Feng J, Xie Y. Privacy-enhanced multi-party deep learning. Neural Networks, 2020, 121: 484-496.
- [9] Hao M, Li H, Xu G, et al. Towards Efficient and Privacy-Preserving Federated Deep Learning[C]//ICC 2019 - 2019 IEEE International Conference on Communications (ICC). IEEE, 2019.DOI:10.1109/ICC.2019.8761267.
- [10] Xu R, Baracaldo N, Zhou Y, et al. HybridAlpha: An Efficient Approach for Privacy-Preserving Federated Learning[J].ACM, 2019.DOI:10.1145/3338501.3357371.
- [11] Zhang, L. Research on Key Issues of Privacy Protection in Federated Learning [Master's thesis, Jiangxi University of Finance and Economics]. DOI: 10.27175/d.cnki.gjxcu.2023.001840
- [12] Wang, S., Jing, T., Xiao, G., et al. (2023). An Efficient Privacy Protection and Secure Aggregation Scheme for Federated Learning. *Computer Systems Application*, 32(11), 175-181. DOI: 10.15888/j.cnki.csa.009302
- [13] Chen, J., Peng, C., Tan, W., et al. (2023). Multi-Server Federated Learning Scheme Based on Differential Privacy and Secret Sharing. *Information Network Security*, 23(07), 98-110.
- [14] He, W. Research on key technology of machine learning based on homomorphic encryption privacy protection [D]. University of Electronic Science and Technology,2019.
- [15] Zhu, Y. (2023). Analysis and Application of Several Loss Functions in Deep Neural Networks [Master's thesis, Strategic Support Force Information Engineering University]. DOI: 10.27188/d.cnki.gzjxu.2023.000063
- [16] Tang, D. Research on Optimization of Federated Learning Algorithms Based on Differential Privacy [Master's thesis, Xi'an University of Technology]. DOI: 10.27398/d.cnki.gxalu.2023.001832
- [17] Song, C., Cheng, D., & Peng, W. (2023). An Efficient Privacy Protection Scheme for Federated Learning. *Journal of Xi'an University of Electronic Science and Technology*, 1-11. Retrieved September 6, 2023, from <http://kns.cnki.net/kcms/detail/61.1076.TN.20230717.2040.002.html>
- [18] Y. Miao and S. Chen, Efficient Privacy-Preserving Federated Learning Against Inference Attacks for IoT, 2023 IEEE Wireless Communications and Networking Conference (WCNC), Glasgow, United Kingdom, 2023, pp. 1-6, doi: 10.1109/WCNC55385.2023.10118613.
- [19] Zhang, H. Local Differential Privacy Design for Federated Learning. *Intelligent Computing and Applications*, 2022(005), 012.
- [20] Wang Z, Yu X, Wang H, et al. A federated learning scheme for hierarchical protection and multiple aggregation[J]. Computers and Electrical Engineering, 2024, 117: 109240.
- [21] Wang Z, Yu X, Huang Q, & Gong Y. "An adaptive differential privacy method based on federated learning," arXiv preprint, arXiv:2408.08909, Aug. 2024.